

Python: Conditionals (條件判斷)

Cheng-Te Li (李政德)

chengte@mail.ncku.edu.tw

Dept. of Statistics, NCKU



Course Info (10/1)

- **Reminder: hand over your HW1 hard copy today or next week**
 - HW1 deadline is extended to 10/2 (Wed.) 11:59
- **Homework 2 (HW2)** is announced today
 - You have 3 weeks for HW2 – Deadline: 10/21 (Mon.) 23:59
 - **Cover Lecture 2, 3, 4 (string, conditionals, and while loop)**
 - You may want to start HW2 from now on
 - Start from what you have learned
- **Programming Exam 1 (PE1)** will be in **late October**
 - The exact date will be announced soon
 - **HW2 is quite relevant PE1 – you better can do HW2 by yourself**
- Teacher will come be here from next week (10/8)
- Concurrently teach
 - Lecture 2: Strings
 - Lecture 3: Conditionals (if-else)
 - Lecture 4: while loop

Boolean Type and Operators

- Python has a **boolean** type called `bool`
- `bool` has only two possible values: `True` and `False`
- In normal speech, “true” and “false” are adjectives, but `True` and `False` are Python values, just as much as the `int` value 0 or the `float` value -17.3
- Only three basic boolean operators, **given in order of precedence (low to high): `or`, `and`, and `not`**
- They have meanings in line with common usage
 - **`and`** and **`or`** are **binary operators**
 - **`not`** is a **unary operator**

Examples of Boolean Operation

```
not True  # False  
not False # True
```

```
False and False  
# => False
```

```
False and True  
# => False
```

```
True and False  
# => False
```

```
True and True  
# => True
```

```
False or False  
# => False
```

```
False or True  
# => True
```

```
True or False  
# => True
```

```
True or True  
# => True
```

Truth Tables

- We define boolean operators with **truth tables**

x	y	x and y	x	y	x or y
False	False	False	False	False	False
False	True	False	False	True	True
True	False	False	True	False	True
True	True	True	True	True	True

and evaluates to **True** only if both its operands are **True**
or evaluates to **True** if either or both of its operands are **True**

Relational Operators

- Most often, **boolean values are created in expressions**
- The most common way is to use **relational operators**

Symbol	Operation
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

Relational Operators

- Relational expressions evaluate to `True` or `False`, just like we can say whether a mathematical statement is true or false
- Note that Python uses `==` for equality (**not** `=`)
- All relational operators are binary: compare two values
- We can compare `integers` to `floats`
 - integers are automatically converted to floats in comparisons

```
45 > 73 # False
```

```
45 > 23 # True
```

```
45 < 73 # True
```

```
45 < 23 # False
```

```
23.1 >= 23 # True
```

```
23.1 >= 23.1 # True
```

```
23.1 <= 23.1 # True
```

```
23.1 < 23 # False
```

```
67.3 == 87 # False
```

```
67.3 == 67 # False
```

```
67.0 == 67 # True
```

```
67.0 != 67 # False
```

```
67.0 != 23 # True
```

```
67 = 23
```

```
>>> 67 = 23
```

```
File "<stdin>", line 1
```

```
SyntaxError: can't assign to literal
```

Relational Operators

- It doesn't make much sense to compare two numbers you know in advance
- Relational operators almost always involve variables

```
a = 123 + 789
b = 456 + 457
print(a > b)           # False
print(a == b)          # False
print(a != b)          # True
print(a == b - 1)      # True, suggest: a == (b - 1)
print((a - 1) >= b)    # False
```

Combining Operators

- **Precedence rules for combining operators**

Arithmetic operators

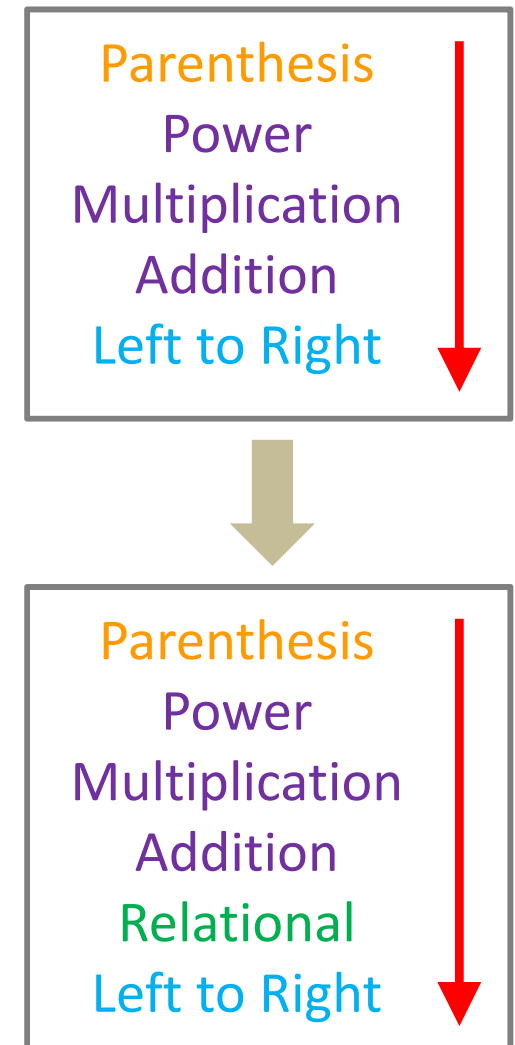
> Relational operators

> Left to right

- For example, + and / are evaluated before < or >

- Also, comparisons are evaluated before **and**, **or**, and **not**

- For example, $1 + 3 > 7$ is evaluated as $(1 + 3) > 7$



Combining Operators

- Often, we may omit the parentheses in complicated expressions
- However, for clarity, we'd rather leave them in

```
x, y, z = 2, 5, 7
x < y and y < z           # True
(x < y) and (y < z)       # True
not (x < z) or (z > y)     # True
not ((x < z) or (z > y))  # False
(not (x < z)) or (z > y)   # True
not (x < z) or not (z > y) # False
```

Range Checking

- We often need to check whether a value lies in a given range
- Python allows chain comparisons

```
x = 3
(1 < x) and (x <= 5)    # True
1 < x <= 5              # True
y = 7
(1 < y) and (y <= 5)    # False
1 < y <= 5              # False
1 < x < y <= 10         # True
```

not Convert Numbers to Boolean

- Python converts **integers** to **floats** in mixed expressions
- Python also converts numbers to **bools**
- **0** and **0.0** are treated as **False**
- All other numbers are treated as **True**

```
not 0          # True
not 1          # False
not 12.3       # False
not -87        # False
```

Logically Equivalent Boolean Expressions

- In numerical algebra, there are arithmetically equivalent expressions of different forms

Logically Equivalent Boolean Expressions

<code>x < y</code>	is equivalent to	<code>not (x >= y)</code>
<code>x <= y</code>	is equivalent to	<code>not (x > y)</code>
<code>x == y</code>	is equivalent to	<code>not (x != y)</code>
<code>x != y</code>	is equivalent to	<code>not (x == y)</code>
<code>not (x and y)</code>	is equivalent to	<code>(not x) or (not y)</code>
<code>not (x or y)</code>	is equivalent to	<code>(not x) and (not y)</code>

Logically Equivalent Boolean Expressions

```
not (10 >= 20)           # True
10 != 20                 # True
not (10 == 20)           # True
not (10 < 20 and 10 < 30) # False
(not 10 < 20) or (not 10 < 30) # False
not (10 < 20 or 10 < 30)  # False
(not 10 < 20) and (not 10 < 30) # False
```

Comparing Strings

- Strings can be compared on their **lexicographic order**
- Uppercase letters come before the lower letters
- If a string s1 is a **prefix** of another, longer string s2, then s1 is “less than” s2

```
'A' < 'a'           # True
'A' < 'z'           # True
'abc' < 'abd'       # True
'abc' < 'abcd'      # True
'abc' == 'abc'      # True
'abc' != 'xyz'      # True
```

`if` statements

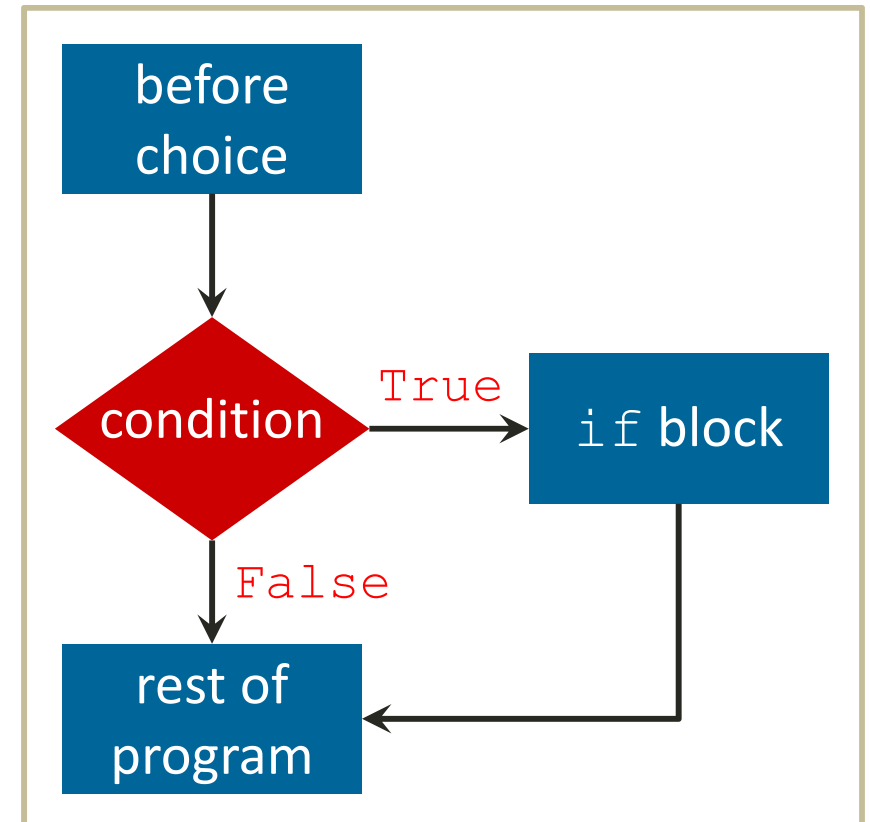
- Use an `if` statement to make a choice

- General form

```
if <condition>:  
    <block>
```


indentation

- If `<condition>` is `True` the `<block>` is executed
- If it is `False`, `<block>` is skipped
- Note that the `<block>` must be **indented**



Condition

- Boolean expressions **as a question** and produce a **True** or **False** result which we can use to control program flow
 - Boolean operators (`and`, `or`, `not`)
 - Relational operators (see below)
- Comparison operators look at variables but do not change the variables

Symbol	Operation
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to
<code>==</code>	Equal to
<code>!=</code>	Not equal to

Code Example

Table: Solution Categories Based on pH Level

pH Level	Solution Category
0–4	Strong acid
5–6	Weak acid
7	Neutral
8–9	Weak base
10–14	Strong base

We can make Python execute certain statements when the pH level represented by some variable falls into a certain category.

Example Code

```
1 ph = float(input("Enter a PH value: "))
2 if ph < 5.0:
3     print("Strong acid")
4 if ph < 7.0:
5     print("Weak acid")
```

```
c:\Python35-32\test>python test.py
Enter a PH value: 5.7
Weak acid
```

- The body of the first if statement is not executed
- But the body of the second one is
- What about a PH of 3.7 ?

```
c:\Python35-32\test>python test.py
Enter a PH value: 3.7
Strong acid
Weak acid
```

What can we do?

Example Code: Using `if`

```
1 ph = float(input("Enter a PH value: "))
2 if 0.0 <= ph < 5.0:
3     print("Strong acid")
4 if 5.0 <= ph < 7.0:
5     print("Weak acid")
```

```
c:\Python35-32\test>python test.py
Enter a PH value: 3.7
Strong acid
```

- This works!
- Or we could use
 - Nested `if`
 - `elif` clause

In-Class Exercise: Grading

- Write a program (grading.py)
 - Allow the user to input a score
 - Print the comment based on the table

```
c:\Python35-32\workspace>python grading.py
Please enter a score: 77
Good

c:\Python35-32\workspace>python grading.py
Please enter a score: 22
Inadequate

c:\Python35-32\workspace>python grading.py
Please enter a score: 99
Outstanding
```

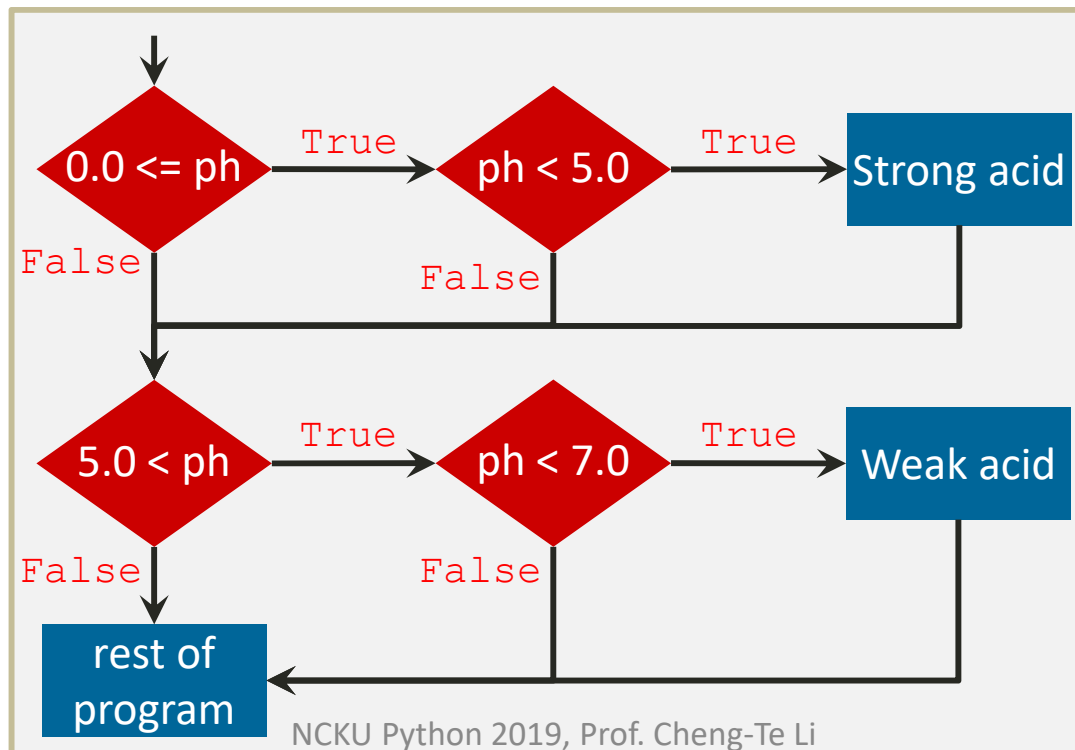
Grade	Comment
90 – 100	Outstanding
80 – 89	Exceptional
70 – 79	Good
60 – 69	Adequate
50 – 59	Marginal
0 – 49	Inadequate

In-Class Exercise: Grading

```
1  score = float(input("Please enter a score: "))
2  if 90 <= score <= 100:
3      print("Outstanding")
4  if 80 <= score < 90:
5      print("Exceptional")
6  if 70 <= score < 80:
7      print("Good")
8  if 60 <= score < 70:
9      print("Adequate")
10 if 50 <= score < 60:
11     print("Marginal")
12 if 0 <= score < 50:
13     print("Inadequate")
```

Example Code: Using Nested `if`

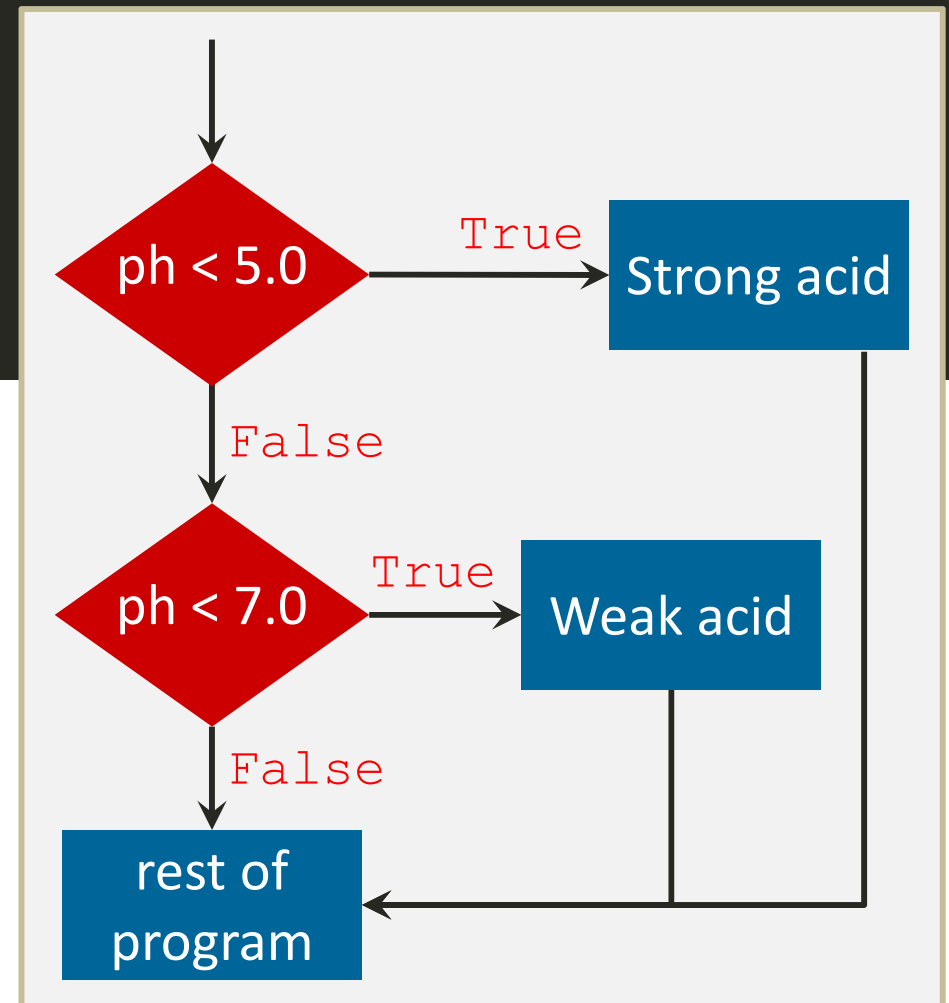
```
1 ph = float(input("Enter a PH value: "))
2 if 0.0 <= ph:
3     if ph < 5.0:
4         print("Strong acid")
5 if 5.0 <= ph:
6     if ph < 7.0:
7         print("Weak acid")
```



Example Code: Using `elif`

```
1 ph = float(input("Enter a PH value: "))
2 if ph < 5.0:
3     print("Strong acid")
4 elif ph < 7.0:
5     print("Weak acid")
```

If the condition of the `if` is `true`, then neither the `elif` nor its block is executed!



elif Clauses

- A condition-and-block pair is called a **clause**
- `elif` is for “else if” `if` **<condition1>**:
<block1>
- General form:
`elif` **<condition2>**:
<block2>
 **<block2>**
- If the **<condition2>** is True, then the **<block2>** is executed;
– if the **<condition2>** is False, then it is skipped
- The **<block>** must be **indented**
- An `elif` may be preceded by other `elif`,
and the top one of these must be preceded by an `if`
- An `if` may be followed by any number (including 0) of `elif`

```
if <condition3>:  
    <block3>  
if <condition4>:  
    <block4>  

```

What will these programs print?

- Or there are some errors?

```
x, y = 7, 16
if x < 5:
    print("hello1")
    if y > 8:
        print("hello2")
elif 5 <= x < 10:
    print("hello3")
    if y > 16:
        print("hello4")
elif x >= 10:
    print("hello5")
print("hello6")
```

hello3
hello6

```
x, y = 2, 5
if x < 5:
    print("hello1")
    elif y > 2:
        print("hello2")
    if y > 8:
        print("hello3")
if x >= 5:
    print("hello4")
elif y > 16:
    print("hello5")
print("hello6")
```

```
File "xxx.py", line 4
    elif y > 2:
        ^
SyntaxError: invalid syntax
```

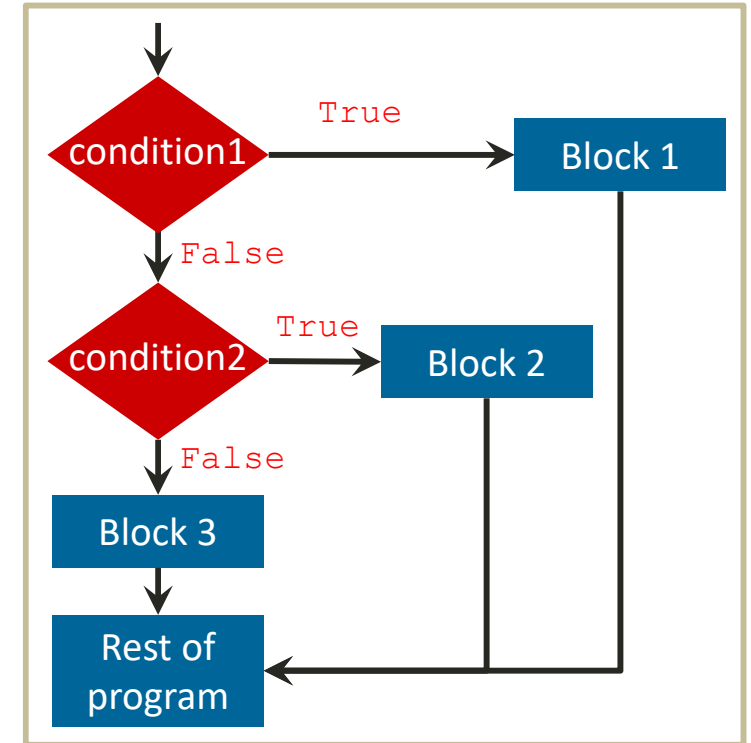
```
x, y, z = 3, 7, 11
if x < 5:
    print("hello1")
    if y > 8:
        print("hello2")
    elif y <= 8:
        print("hello3")
elif z >= 10:
    print("hello4")
    elif y > 16:
        print("hello5")
print("hello6")
```

```
File "xxx.py", line 10
    elif y > 16:
        ^
SyntaxError: invalid syntax
```

else Clause

- General form:

```
if <condition1>:  
    <block1>  
elif <condition2>:  
    <block2>  
else:  
    <block3>  
    {  
    indentation
```



- An `if` statement can have at most one `else` clause
 - An `if` statement can have neither `elif` nor `else`
- The `else` clause must be the final clause
- Note that:

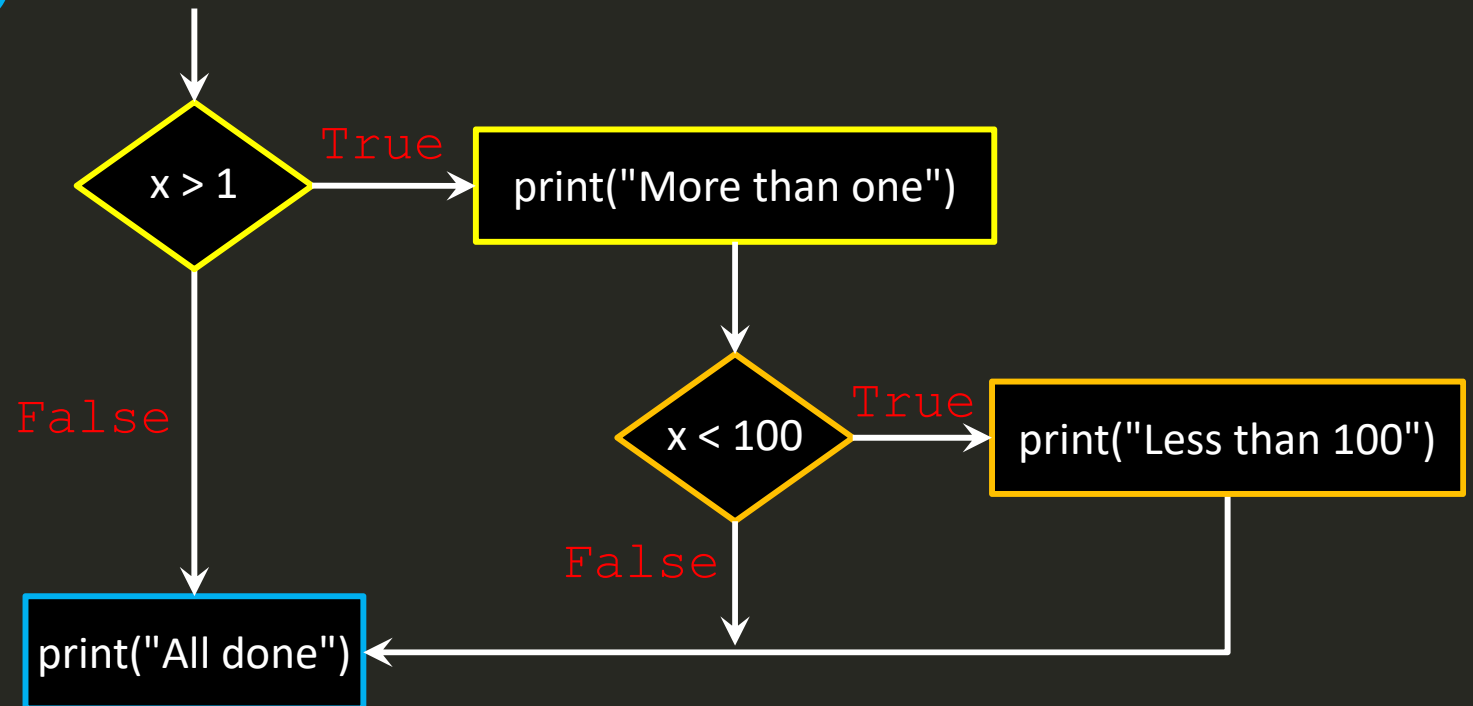
```
if <condition1>:  
    <if block>  
else:  
    <else block>
```

is equivalent to

```
if <condition1>:  
    <if block>  
if not <condition1>:  
    <else block>
```

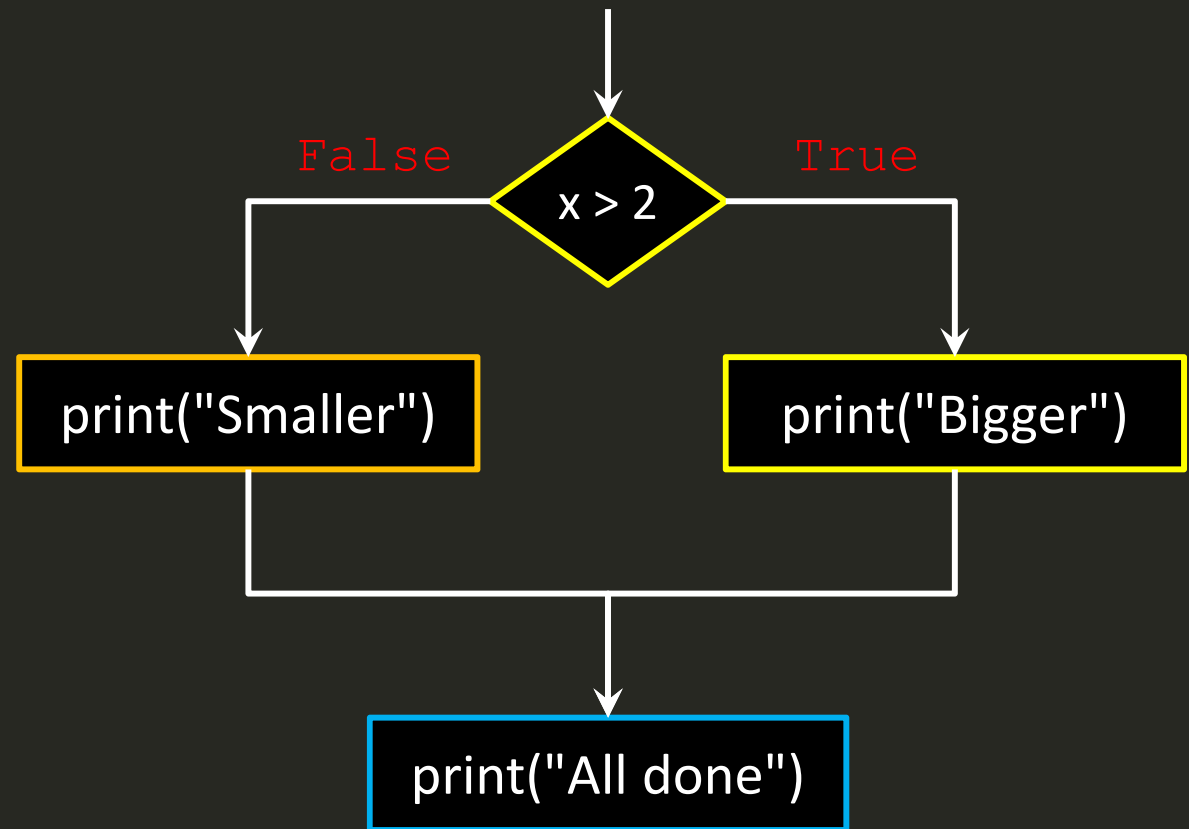
Nested Decisions

```
x = 42
if x > 1:
    print("More than one")
    if x < 100:
        print("Less than 100")
print("All done")
```



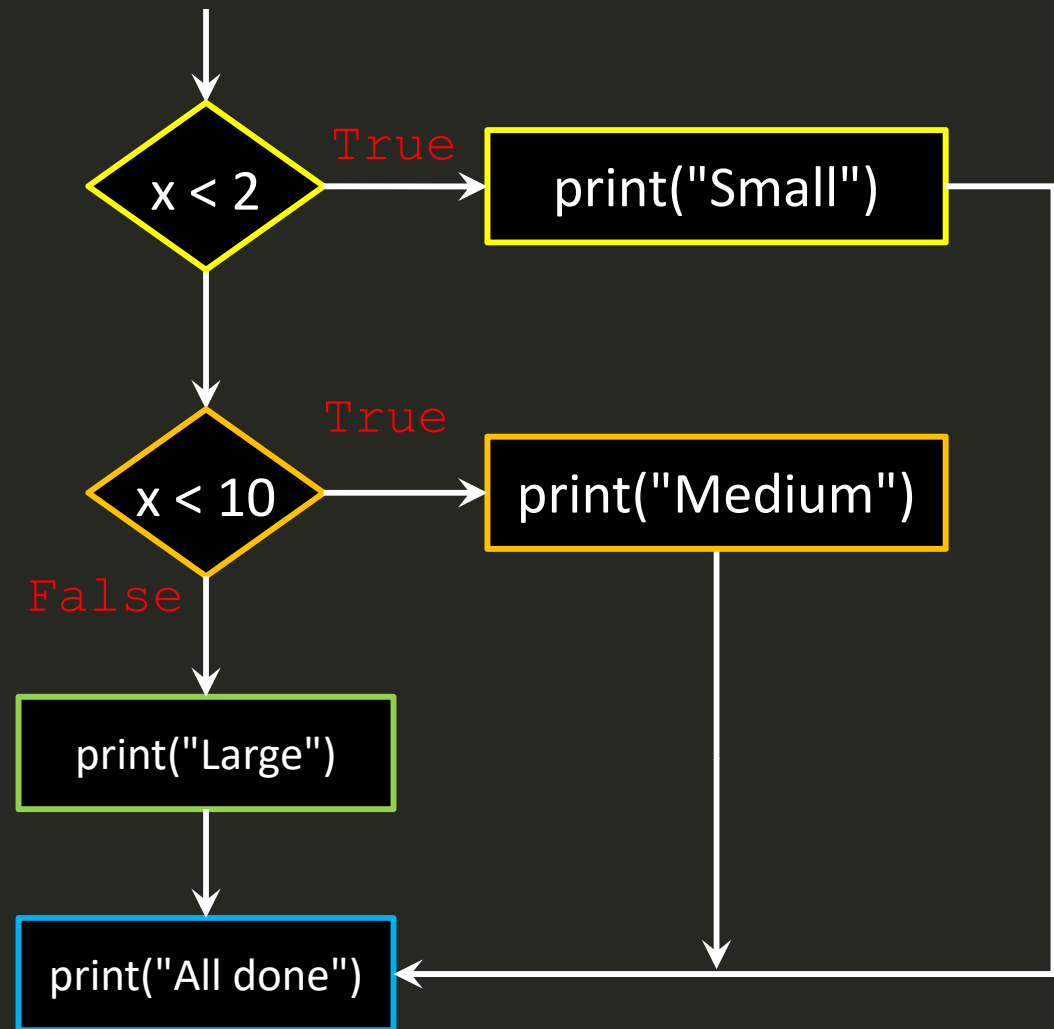
Two-way Decisions

```
x = 4
if x > 2:
    print("Bigger")
else:
    print("Smaller")
print("All done")
```



Multi-way Decisions

```
x = 0
if x < 2:
    print("Small")
elif x < 10:
    print("Medium")
else:
    print("LARGE")
print("All done")
```



Multi-way Puzzles

- What will never print?

```
1 if x < 2:
2     print("Below 2")
3 elif x >= 2:
4     print("Two or more")
5 else:
6     print("Something else")
```

```
1 if x < 2:
2     print("Below 2")
3 elif x < 20:
4     print("Below 20")
5 elif x < 10:
6     print("Below 10")
7 else:
8     print("Something else")
```

Indentation

- **Increase indent** after an `if` statement or `while` and `for` loop statements (after `:`)
- **Maintain indent** to indicate the **scope** of the **block** (which lines are affected by the `if/for`)
- **Reduce indent** *back* to the level of the `if` statement or `while/for` statement to indicate the end of the block
- **Blank lines** are ignored - they do not affect **indentation**
- **Comments** on a line by themselves are ignored with regard to **indentation**

Generally we use **TAB** for indentation!

increase / maintain indent after if or for
decrease indent to indicate end of block

```
1  x = 5
2  if x > 2 :
3      print("Bigger than 2")
4      print("Still bigger")
5  print("Done with 2")
6
7  for i in range(5):
8      # here is in a for loop
9      print(i)
10     if i > 2:
11         print("Bigger than 2")
12
13     print("Done with i", i)
14 print("All Done")
```

Think about Begin/End of blocks

```
1  x = 5
2  if x > 2 :
3      print("Bigger than 2")
4      print("Still bigger")
5  print("Done with 2")
6
7  for i in range(5):
8      # here is in a for loop
9      print(i)
10     if i > 2:
11         print("Bigger than 2")
12
13     print("Done with i", i)
14 print("All Done")
```

block

block

block

In-Class Exercise

```
1 myinput = input("pH value: ")
2 if len(myinput) > 0:
3     ph = float(myinput)
4     if ph < 0.0 or ph > 14.0:
5         print("Invalid pH value")
6     elif 0 <= ph < 7.0:
7         print("Acidic")
8     elif 7.0 < ph <= 14.0:
9         print("Basic")
10    else:
11        print("Neutral")
12 else:
13    print("No pH value given")
```

Your task: modify the code to fit this table

pH Level	Solution Category
0 <= pH < 5	Strong acid
5 <= pH < 7	Weak acid
7 <= pH < 8	Neutral
8 <= pH < 10	Weak base
10 <= pH < 15	Strong base

In-Class Exercises

- Write a program that convert temperatures (Celsius/Fahrenheit) selected and entered by users

$$^{\circ}\text{C} = (^{\circ}\text{F} - 32) \times 5 / 9$$

$$^{\circ}\text{F} = (^{\circ}\text{C} \times 9 / 5) + 32$$

```
1  # Temperature Conversion Program (Celsius-Fahrenheit / Fahrenheit-Celsius)
2
3  # Display program welcome
4  print("This program will convert temperatures (Celsius/Fahrenheit)")
5  print("Enter (F) to convert Fahrenheit to Celsius")
6  print("Enter (C) to convert Celsius to Fahrenheit")
7
8  # Get temperature to convert
9  which = input("Enter selection: ")
10 temp = int(input("Enter temperature to convert: "))
11
12 # Determine temperature conversion needed and display results
13 if which == "F":
14     converted_temp = (temp - 32) * 5.0 / 9.0
15     print(temp, "degrees Fahrenheit equals", converted_temp, "degrees Celsius")
16 else:
17     if which == "C":
18         converted_temp = (9.0 / 5.0 * temp) + 32
19         print(temp, "degrees Celsius equals", converted_temp, "degrees Fahrenheit")
20     else:
21         print("INVALID INPUT!")
```

In-Class Exercise: Risk of Heart Disease

- Write a program (bmiage.py) that allows users to
 - Input his/her BMI value and age
 - Print out his/her risk of heart disease based on the following table

BMI	Age	
	< 45	≥ 45
< 22	Low	Medium
≥ 22	Medium	High

Figure: Risk of heart disease, based on age and body mass index

```
c:\Python35-32\workspace>python bmiage.py
Enter your age: 30
Enter your BMI: 25
Your risk of heart disease is medium
```

```
c:\Python35-32\workspace>python bmiage.py
Enter your age: 50
Enter your BMI: 50
Your risk of heart disease is high
```

In-Class Exercise: Risk of Heart Disease

```
1 age = int(input("Enter your age: "))
2 bmi = float(input("Enter your BMI: "))
3 if age < 45:
4     if bmi < 22.0:
5         risk = 'low'
6     else:
7         risk = 'medium'
8 else:
9     if bmi < 22.0:
10        risk = 'medium'
11    else:
12        risk = 'high'
13 print("Your risk of heart disease is", risk)
```

BMI	Age	
	< 45	≥ 45
< 22	Low	Medium
≥ 22	Medium	High

Alternative Solution with Stored Conditionals

```
1 age = float(input("Your Age: "))
2 bmi = float(input("Your BMI: "))
3 young = age < 45
4 slim = bmi < 22.0
5 if young and slim:
6     risk = 'low'
7 elif young and not slim:
8     risk = 'medium'
9 elif not young and slim:
10    risk = 'medium'
11 elif not young and not slim:
12    risk = 'high'
13 print(risk)
```

BMI	Age	
	< 45	≥ 45
< 22	Low	Medium
≥ 22	Medium	High

Stored Conditionals as List Indices

```
1 age = float(input("Your Age: "))
2 bmi = float(input("Your BMI: "))
3 table = [['medium', 'high'], ['low', 'medium']]
4 young, heavy = age < 45, bmi >= 22.0
5 risk = table[young][heavy]
6 print(risk)
```

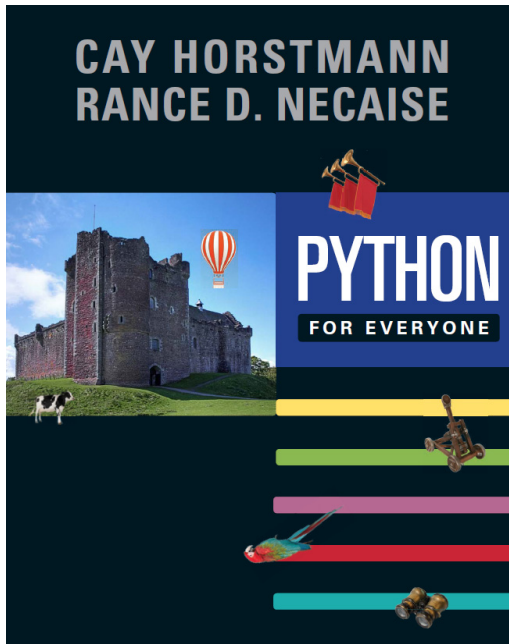
	0	1
0	'medium'	'high'
1	'low'	'medium'

BMI	Age	
	< 45	≥ 45
	Low	Medium
< 22	Low	Medium
≥ 22	Medium	High

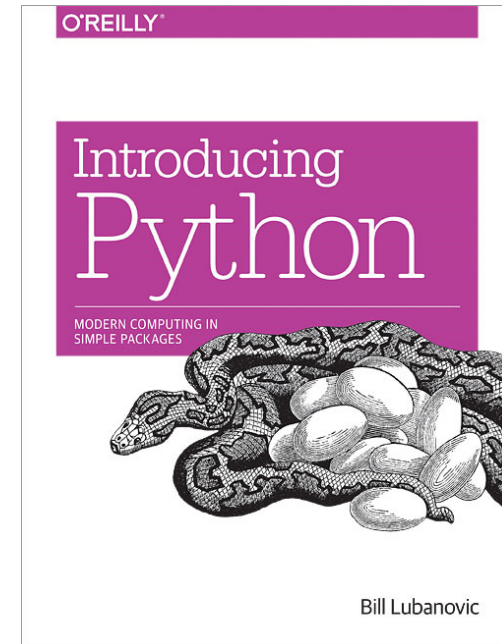
Summary

- Logic Operator
- Relational Operator
- `if`
- `elif`
- `else`
- Nested, Two-way, Multi-way choices

Suggested Reading



P.91 – P.135



P.69 – P.74